C# Programming Exercise 4: Lists and Generics

Overview

This activity will help you learn to work with Lists and Generics in C#.

Preparation

Collections

The core C# libraries provide all of the standard collections and data structures you are likely to need in your programs, including Lists, Sets, Dictionaries, and so forth. This article will focus on lists, but you can easily look up the information for any other data structure you might need.

Lists

As you know, a major difference between C# and Python is that you must declare your variable types in C#. While at first, this may seem like a burden in C#, you'll soon discover that it helps you avoid many runtime errors.

In a similar way, when you declare a new list variable in C#, you not only declare that it is a **`List`**, you must also declare the type of data that can be put in the list. That way, if you create a list of users, you will be prevented from accidentally adding an order or a product variable to this list.

To create a new list of integers, you specify **`int`** inside angle brackets **<>** directly following the name of the data structure: **`List<int>`** and if you want to have a list of strings, you would use: **`List<string>`** as shown below.

Generic Data Types

Rather than having to implement separate code for the data structure for every kind of data that could ever be put into it, the developers of the C# List library make use of a concept called "Generics."

With generics, the library is written *generically* and then the type can be provided to fill in the template. In documentation, you'll often see `T` used as the template value to be filled in later.

```
List<int> numbers;
List<string> words;
```

The code above declares a variable to hold the list, but before you can use one, you need to create a **new** one to use with the `new` keyword.

```
List<int> numbers;
numbers = new List<int>();
```

This is typically done on the same line:

```
List<int> numbers = new List<int>();
List<string> words = new List<string>();
```

Notice the extra parentheses `()` at the end, that we use any time we create a new object.

One more important thing to be aware of: Any file that uses Lists (or any other standard collection), must refer to that library at the top of the file. (This is so common that sometimes your settings for C# can be specified so that you do not not have include this, but it is important to know about it, in case you run into problems.)

```
using System.Collections.Generic;
```

What is "new" and why do we need it?

It turns out that **`List`** is a *class* or custom data type and we are creating a new object or instance of that class. This is actually the complete focus of this course, and beginning next week you will learn how to create your very own custom classes.

With this in mind, you will learn much more about this in coming weeks, but for now, just remember to include **`new`** before you start using a list.

Adding Items to the List

To add items to the list, you use the **`.Add()`** method:

C#

```
using System.Collections.Generic;

...

List<string> words = new List<string>();

words.Add("phone");
words.Add("keyboard");
words.Add("mouse");
```

Python

```
words = []

words.append("phone")
words.append("keyboard")
words.append("mouse")
```

Getting the list size

You can get the size of the list with the **`Count`** property.

C#

```
Console.WriteLine(words.Count);
```

Python

```
print(len(words))
```

Notice that you do not include parentheses `()` after `Count` because it is a property, not a function.

Iterating through a List

The easiest (and safest) way to iterate through a list in C# is to use the `foreach` loop:

C#

```
foreach (string word in words)
{
    Console.WriteLine(word);
}
```

Python

```
for word in words:
    print(word)
```

You can also access the items by their index:

C#

```
for (int i = 0; i < words.Count; i++)
{
    Console.WriteLine(words[i]);
}
```

Python

```
for i in range(len(words)):
    print(words[i])
```

Other Operations

There are many other things you can do with lists. You can view the official documentation or also, just begin typing in VS Code and see the options that the Intellisense editor pops up for you.

In addition, don't forget that you can easily find syntax with a quick Web search!

Assignment Instructions

For this assignment, you will complete another assignment that you did previously in CSE 110, but in this case, **write the program in C#**:

Program Specification

Here are the instructions that you saw previously in CSE 110 that we will use as our program specification:

> ### Assignment
>
> Ask the user for a series of numbers, and append each one to a list. Stop when they enter `0`. (**Remember**: You should not add 0 to the list. If you do, later calculations and operations will not be correct.)
>
> Once you have a list, have your program do the following:
>
> ### Core Requirements

Work through these core requirements step-by-step to complete the program. Please don't skip ahead and do the whole thing at once, because others on your team may benefit from building the program up slowly.

1. Compute the sum, or total, of the numbers in the list.
2. Compute the average of the numbers in the list.
3. Find the maximum, or largest, number in the list.

The following shows the expected output:

```
Enter a list of numbers, type 0 when finished.
Enter number: 18
Enter number: 36
Enter number: 2
Enter number: 48
Enter number: 6
Enter number: 12
Enter number: 9
Enter number: 0
The sum is: 131
The average is: 18.714285714285715
The largest number is: 48
```

Stretch Challenge

1. Have the user enter both positive and negative numbers, then find the smallest positive number (the positive number that is closest to zero).
2. Sort the numbers in the list and display the new, sorted list. Hint: There are C# libraries that can help you here, try searching the internet for them.

The following shows the expected output after completing the stretch challenges:

```
Enter a list of numbers, type 0 when finished.
Enter number: 3
Enter number: 5
Enter number: 7
```

```
Enter number: 3
Enter number: 2
Enter number: -1
Enter number: -4
Enter number: -8
Enter number: 0
The sum is: 7
The average is: 0.875
The largest number is: 7
The smallest positive number is: 2
The sorted list is:
-8
-4
-1
2
3
3
5
7
```

Write and Test the program

1. Write and test the program as described above.
2. You should complete the 3 Core Requirements.
3. The stretch challenges are optional.
4. Make sure to use the same project template that you did for the previous activities. However, this time, you should add your code in **`Program.cs`** file in the the **`Exercise4`** project.

Sample Solution

When you have finished the program, please compare your approach to the one from this sample solution:

- C# Programming Exercise 04 Sample Solution

Getting Help

If you get stuck, please ask questions in MS Teams.

Submitting

Return to Canvas and complete the corresponding quiz.